

CAPÍTULO 3

ARCHIVOS



3.1. Lección 10

En esta lección, agregaremos un nuevo recurso: archivos, a través de los cuales podremos grabar información que quedará almacenada en el disco rígido o cualquier otro medio de almacenamiento. También mostraremos cómo acceder a información que previamente fue grabada por otros programas.

3.1.1. Introducción

3.1.1.1. Archivo

Un archivo representa un conjunto de bytes almacenados en algún medio de almacenamiento, uno a continuación del otro. Los programas pueden usar archivos para guardar la información que generan. Posteriormente, esta misma información podrá ser utilizada (accedida) por otros programas.

Los datos que guardamos en un archivo son persistentes, esto significa que persisten luego de que haya finalizado el programa que los generó.



Lección 10

3.1.1.2. Tipos de archivo

Habitualmente, cuando trabajamos con la computadora utilizamos archivos de diferentes tipos, que se identifican por la extensión de su nombre (últimas tres letras). Por ejemplo: `txt`, `docx`, `pdf`, `mp3` y `avi` son algunas de las extensiones que se utilizan para identificar cuál es el contenido de un archivo y cómo está distribuida la información dentro de él.

Sin embargo, independientemente de cuál sea su tipo, todos los archivos se componen de un conjunto de bytes almacenados uno a continuación del otro en algún dispositivo de almacenamiento, y se identifican por un nombre (*filename*). La extensión (últimas tres letras del *filename*) sólo es una referencia que ayuda a identificar qué tipo de información contiene el archivo en cuestión.

3.1.1.3. Archivos binarios

Todos los archivos son binarios, pues se trata de un conjunto de bytes almacenados en algún dispositivo.

Por ejemplo: un archivo con los siguientes 3 bytes es un archivo binario.

Binario	00110111	11000100	11111010
Hexadecimal	37	C4	FA
Decimal	55	196	250

Tabla 3.1. Archivo de 3 bytes

3.1.1.4. Archivos de texto

Si todos los bytes que componen un archivo tienen representación ASCII, decimos que se trata de un *archivo de texto*. En otras palabras, un archivo de texto es un archivo binario que cumple con la condición recién mencionada.

Por ejemplo: un archivo con los siguientes 3 bytes es un archivo de texto.

Binario	01000001	01000010	01000011
Hexadecimal	41	42	43
Decimal	65 (A)	66 (B)	67 (C)

Tabla 3.2. Archivo de texto de 3 bytes

Observemos que los bytes del archivo anterior corresponden a los números decimales 65, 66 y 67; códigos ASCII de los caracteres A, B y C, respectivamente.

En este curso, sólo trabajaremos con archivos binarios.

3.1.2. Gestión de archivos

3.1.2.1. Funciones de biblioteca

Los lenguajes de programación proveen funciones para manipular archivos, a través de las cuales podemos crear nuevos archivos, borrar los existentes, leer y modificar su contenido, y otras funcionalidades que iremos estudiando.

En C y C++ las principales funciones de gestión de archivos son:

<code>FILE*</code>	Tipo de dato “archivo”.
<code>fopen</code>	Abre un archivo y retorna un valor tipo <code>FILE*</code> asociado al archivo que se abrió.
<code>fwrite</code>	Graba datos en un archivo nuevo o modifica los datos de un archivo existente.
<code>fread</code>	Lee datos desde un archivo.
<code>feof</code>	Indica si se llegó al final de un archivo.
<code>fseek</code>	Modifica el valor del indicador de posición de un archivo.
<code>ftell</code>	Indica cuál es el valor actual del indicador de posición.
<code>fclose</code>	Cierra un archivo que previamente fue abierto con <code>fopen</code> .

Tabla 3.3. Tipo y funciones para manejo de archivos

Estas funciones están declaradas en la biblioteca `stdio.h`, y para utilizarlas debemos agregar la siguiente al inicio del programa: `#include<stdio.h>`.

3.1.2.2. Grabar y leer datos

En el siguiente ejemplo, a la izquierda, vemos cómo crear un archivo y grabar caracteres en él; a la derecha, cómo leer su contenido y mostrarlo por pantalla.

Grabar un archivo

```
// abro el archivo para
FILE* f=fopen("c.xx", "w+b");
```

Leer un archivo

```
// abro para lectura
FILE* f=fopen("c.xx", "r+b");
```

```
// grabo un valor
char c = 'A';
fwrite(&c, sizeof(char), 1, f);

// grabo otro valor
c = 'B';
fwrite(&c, sizeof(char), 1, f);

// grabo otro valor
c = 'C';
fwrite(&c, sizeof(char), 1, f);

// grabo otro valor mas
c = 'D';
fwrite(&c, sizeof(char), 1, f);

// cierro el archivo
fclose(f);
```

```
char c;

// leo el primer byte
fread(&c, sizeof(char), 1, f);

// mientras no llegue el fin
while( !feof(f) )
{
    // caracterleido
    cout << c << endl;

    // leo caracter
    fread(&c, sizeof(char), 1, f);
}

// cierro el archivo
fclose(f);
```

En `c.xx` grabamos cuatro caracteres: A, B, C, D, cuyos códigos ASCII son: 65, 66, 67 y 68, que equivalen a: 01000001|01000010|01000011|01000100.

Como todos los bytes del archivo tienen representación ASCII y la unidad de información es 1 byte, decimos que `c.xx` es un archivo de texto, independientemente de que su extensión no sea `txt`.

En estos ejemplos, utilizamos las siguientes funciones de biblioteca:

`fwrite`, cuyos parámetros son:

- Un puntero al *buffer* (variable) que contiene el dato que grabaremos.
- El tamaño (en bytes) del tipo de dato de dicho *buffer*, que se obtiene invocando a la función: `sizeof`.
- La cantidad de unidades de información del tamaño antes indicado que queremos escribir en el archivo; en nuestro caso: 1.
- El archivo donde se grabará el contenido almacenado en el *buffer*.

`fopen`, cuyos parámetros son dos cadenas:

- El nombre físico del archivo que abriremos.
- La modalidad de apertura, que puede ser: “w+b” o “r+b”.

`fclose`, que cierra el archivo que recibe como parámetro.

`fread`, que recibe exactamente los mismos parámetros que `fwrite`.

Para abrir un archivo hay que invocar a la función `fopen`. Si usamos la modalidad de apertura “w+b”, `fopen` creará un nuevo archivo; si el archivo ya existía lo borrará y creará uno nuevo (vacío). En cambio, si usamos “r+b” estaremos indicando que queremos leer el archivo; por lo tanto, el archivo debe existir con antelación.

3.1.2.3. Archivo secuencial

En el programa de la izquierda (que analizamos más arriba), grabamos los cuatro caracteres de modo secuencial, es decir: cada carácter se escribe a continuación del anterior, aumentando de a 1 byte la longitud total del archivo.

Por su parte, en el programa de la derecha invocamos a `fread` antes de ingresar al `while` y también justo antes de cerrar el ciclo de repeticiones. Cada lectura accede y retribuye el siguiente byte, comenzando con el primero, luego el segundo, así hasta que hayamos leído todo el contenido del archivo.

La función `feof`, que usamos en la condición del ciclo `while`, retorna `false` o `true` según queden o no más bytes por ser leídos. El acrónimo EOF se forma con las iniciales de *End Of File*, que en español significa: Fin Del Archivo.

Internamente, dentro del tipo `FILE*` se guarda un contador llamado *indicador de posición* que hace referencia al próximo byte que debe ser leído o escrito. Las funciones `fread` y `fwrite` aumentan el valor de dicho contador de modo tal que nunca `fread` leerá dos veces la misma información, ni `fwrite` reescribirá (pisará) la información que acabó de escribir.

La función `ftell` retorna el valor actual del indicador de posición del archivo, y la función `fseek` permite que le asignemos un valor arbitrario.

3.1.2.4. Archivos de registros de longitud fija

Cuando podemos identificar que existe un determinado patrón entre los bytes que componen el archivo, lo llamamos *registro*. Si este patrón se repite a lo largo de todo el contenido del archivo, diremos que se trata de un *archivo de registros*.

Los archivos de texto son archivos de registros, donde cada registro contiene un carácter. Dicho de otro modo, cada 8 bits de información se representa un carácter. El patrón se repite cada 8 bits o 1 byte.

Si en un archivo grabamos datos tipo `short`, cada 2 bytes tendremos un número entero. También será un archivo de registros donde cada registro es un valor numérico entero representado en 2 bytes de longitud.

Grabar un archivo de `short`

```
// abro el archivo para
FILE* f=fopen("s.xx","w+b");

// grabo un valor
short s = 1234;
fwrite(&s,sizeof(short),1,f);

// grabo otro valor
s = 4321;
fwrite(&s,sizeof(short),1,f);

// grabo otro valor
s = -9876;
fwrite(&s,sizeof(short),1,f);

// cierro el archivo
fclose(f);
```

Leer un archivo de `short`

```
// abro para lectura
FILE* f=fopen("s.xx","r+b");

short s;

// leo el primer byte
fread(&s,sizeof(short),1,f);

// mientras no llegue el fin
while( !feof(s) )
{
    // caracterleido
    cout << s << endl;

    // leo caracter
    fread(&s,sizeof(short),1,f);
}

// cierro el archivo
fclose(f);
```

La longitud del archivo `s.xx` será de 6 bytes, ya que contiene 3 `short` y cada uno representa 2 bytes.

3.1.2.5. **Big-endian y Little-endian**

Ambos términos hacen referencia al modo en que los datos de más de 1 byte serán almacenados en el archivo. *Big-endian* es el formato natural e indica que los bytes más significativos aparecen primero. Si se utiliza este formato los datos quedarán almacenados exactamente como lo imaginamos. En cambio, si se utiliza el formato *Little-endian* los datos se almacenarán al revés, es decir, los bytes menos significativos aparecerán a la izquierda.

Veamos cómo quedaría el archivo `s.xx`, que generamos con el programa anterior, almacenado en cada uno de estos formatos.

<i>Big-endian</i>	00000100 11010010 00010000 11100001 10100110 10010100
<i>Little-endian</i>	11010010 00000100 11100001 00010000 10010100 10100110

Tabla 3.4. *Big-endian*, *Little-endian*

El modo en que se grabarán los datos dependerá de la arquitectura del sistema operativo para el cual estemos compilando el programa. De todos modos, para nosotros (como programadores), la mayoría de las veces esto será irrelevante.

No importa si `fwrite` graba los datos en formato *Big-endian* o *Little-endian*; `fread` sabrá interpretarlo y los leerá sin problema.

3.1.2.6. Archivos de registros de longitud variable

Los archivos `c.xx` y `s.xx`, que contienen datos tipo `char` y `short` respectivamente, son archivos de registros de longitud fija, pues todos sus registros tienen exactamente el mismo formato y longitud. Sin embargo, existen archivos de registros de longitud variable. Por ejemplo, pensemos en la siguiente estructura:

```
struct Persona
{
    int dni;
    string nombre
};
```

Si en un archivo grabamos registros tipo `Persona`, ¿todos tendrán la misma cantidad de bytes, o dependerá de la longitud de la cadena que le asignemos al campo `nombre` de cada uno?

En esta obra, sólo trabajaremos con archivos de registros de longitud fija, aunque en los ejercicios de este capítulo sí analizaremos un ejercicio sobre *archivo de registros de longitud variable*.

3.1.2.7. Archivos de estructuras

Para grabar estructuras en un archivo, el procedimiento es exactamente el mismo que usamos para grabar caracteres y números enteros. Aunque debemos tener en cuenta que la estructura no puede tener campos tipo `string`, justamente, porque su longitud es variable y los registros ya no tendrían una longitud fija.

La solución a este problema consiste en utilizar *arrays* de caracteres, al estilo del *lenguaje de programación C*. A pesar de que aún no hemos estudiado *arrays*, podemos adelantar algunas cuestiones que debemos tomar en cuenta para reemplazar `string` por `char[]` (léase *char array*, *char corchete* o *array de caracteres*).

La estructura `Persona` que expusimos previamente debería modificarse así:

```
struct Persona
{
    int dni;
    char nombre[15];
};
```

Ahora el campo `nombre` es un *array* que puede contener hasta 15 caracteres. Sin embargo, independientemente de cuántos caracteres tenga el valor que le asignemos, su longitud siempre será fija: 15 bytes.

Como trabajamos con `string`, necesitaremos conocer algunos mecanismos que nos permitan convertir entre `string` y `char[]`, y viceversa.

Veamos cómo pasar de `string` a `char[]`.

```
// un string
string s = "Hola";

// declaro un char array
char c[10];

// asigna el contenido de s a c
strcpy(c,s.c_str());

// muestra: Hola
cout << c << endl;
```

La función `strcpy` es parte de la biblioteca del lenguaje C. Para usarla debemos agregar la línea `#include <string.h>` en el encabezado del programa.

La conversión entre `char[]` y `string` es directa:

```
char nom[10];
// :
string s = nom;
```

En el caso de las estructuras, aun cuando alguno de sus campos sea tipo `char[]` podremos utilizar `{ }` (llaves) para darles valor inicial.

```
Persona p = {10, "Pablo"};
```

Ahora sí, estamos en condiciones de grabar personas en un archivo.

```
FILE* f = fopen("p.xx", "w+b");
Persona p = {10, "Pablo"};
fwrite(&p, sizeof(Persona), 1, f);
p = {20, "Juan"};
fwrite(&p, sizeof(Persona), 1, f);
p = {30, "Pedro"};
fwrite(&p, sizeof(Persona), 1, f);
fclose(f);
```

El siguiente programa lee los registros tipo `Persona` que grabamos en `p.xx`.

```
FILE* f = fopen("p.xx", "r+b");
Persona p;
fread(&p, sizeof(Persona), 1, f);
while( !feof(f) )
{
    cout << personaToString(p) << endl;
    fread(&p, sizeof(Persona), 1, f);
}
fclose(f);
```

La función `personaToString` la resolvemos así:

```
string personaToString(Persona p)
{
    string nom = p.nombre;
    return intToString(p.dni)+", "+nom;
}
```

3.1.2.8. Posicionamiento directo

La función de biblioteca `fseek` permite manipular arbitrariamente el indicador de posición del archivo. Por ejemplo, en el siguiente código usamos `fseek` para mover el indicador de posición de `c.xx` (que contiene caracteres) al byte número 2, que es el tercer byte del archivo; pues el primero ocupa la posición 0.

```
FILE* f = fopen("c.xx", "r+b");
fseek(f, 2, SEEK_SET);

char c;
fread(&c, sizeof(char), 1, f);
```

La constante `SEEK_SET` indica que el valor 2, que le pasamos como argumento a `fseek`, es un valor absoluto, contando desde el inicio del archivo.

Para posicionarnos en el último byte del archivo, leerlo, y posteriormente mostrarlo por consola, usamos la constante `SEEK_END`.

```
FILE* f = fopen("c.xx", "r+b");
fseek(f, -1, SEEK_END);

char c;
fread(&c, sizeof(char), 1, f);
```

En este caso, `SEEK_END` indica que el segundo argumento (-1) hace referencia al final del archivo. De este modo, -1 es la posición que ocupa el último byte.

Por último, la constante `SEEK_CUR` permite especificar, a partir de la posición actual del archivo, cuántos bytes queremos desplazar el indicador de posición. Por ejemplo, en el siguiente código movemos el indicador de posición 1 byte hacia atrás (hacia el inicio del archivo); y lo leemos.

```
// :
fseek(f, -1, SEEK_CUR);
fread(&c, sizeof(char), 1, f);
// :
```

La siguiente tabla resume el significado de cada una de estas constantes.

Constante	Descripción
<code>SEEK_SET</code>	Indica que el número de byte que especifiquemos como argumento de <code>fseek</code> será relativo al inicio del archivo. Con esta constante, el byte 0 será el primer byte del archivo, 1 el segundo y así sucesivamente.
<code>SEEK_END</code>	Si usamos esta constante, el número de byte al que hagamos referencia será considerado a partir del final del archivo. Así, el byte 0 será <i>eof (end of file)</i> , -1 el último, -2 el anteúltimo y así sucesivamente.
<code>SEEK_CUR</code>	Esta constante indica que el número de byte que especifiquemos será relativo a la posición actual (<i>current</i>) del archivo. Así, el byte 0 será la posición actual, 1 será el siguiente byte y -1 será el byte anterior.

Tabla 3.5. *Big-endian, Little-endian*

3.1.2.9. Posicionamiento directo en registros

Ya grabamos tres valores tipo `short` en el archivo `s.xx`, y vimos que cada uno ocupa 2 bytes; razón por la cual el archivo tiene una longitud total de 6 bytes.

El primer número que grabamos (1234) quedó almacenado en los primeros 2 bytes del archivo (bytes 0 y 1). Los siguientes 2 bytes (bytes 2 y 3) contienen el valor del segundo número (4321) y los 2 últimos bytes reflejan el tercer valor (-9876).

0	1	2	3	4	5
00000100 11010010		00010000 11100001		10100110 10010100	
1234		4321		-9876	

Figura 3.1. Contenido del archivo s.xx

Aunque con `fseek` nos podemos posicionar en cualquier byte del archivo, si éste contiene registros de longitud fija es probable que queramos posicionarnos justo al inicio de alguno de estos registros, cuya posición (número de byte) coincidirá con algún múltiplo de dicha longitud.

En nuestro caso, según se muestra en la figura anterior, podríamos posicionarnos en los bytes 0, 2 o 4, pues estos bytes marcan el inicio de un nuevo registro. Justamente, estas posiciones coinciden con el producto entre el número de registro que comienza y el tamaño del tipo de dato de los registros del archivo.

En el siguiente código, nos posicionamos al inicio del tercer registro del archivo (recordemos que se numeran desde cero) y leemos su contenido.

```
int recNo = 2;
fseek(f, recNo*sizeof(short), SEEK_SET);
fread(&c, sizeof(short), 1, f);
```

3.1.2.10. Eliminar registros físicamente

Como los archivos se componen de un conjunto de bytes consecutivos, no hay manera de borrar registros sin que esto implique reescribir todo el archivo. De hecho, el único modo sería creando un nuevo archivo que omita aquellos registros que queremos quitar. En cualquier caso, si el archivo es grande, esta operación llevaría demasiado tiempo de procesamiento, por lo que generalmente no es una opción válida.

3.1.2.11. Eliminar registros lógicamente

Esta operación se llama *baja lógica*, porque anula aquellos registros que no queremos volver a ver. Físicamente, seguirán estando allí, pero tendrán una marca que nos indicará que ya no forman parte de nuestro conjunto de datos.

Trabajaremos con la siguiente estructura, aceptando que `Fecha` es un TAD.

```

struct Estudiante
{
    int legajo;
    Fecha fecUltExamen; // TAD Fecha
    char nombre[15];
    int bajaLogica;      // 0=>activo, 1=>eliminado
};

```

En el siguiente programa, damos de baja a todos aquellos estudiantes que llevan dos o más años sin haber rendido un examen.

```

int main()
{
    FILE* f = fopen("estudiantes.dat","r+b");
    Estudiante e;

    // leo el primer estudiante
    fread(&e,sizeof(Estudiante),1,f);
    while( !feof(f) )
    {
        if( fechaAniosTranscurridos(e.fecUltExamen)>=2 )
        {
            // tenemos que eliminar a este estudiante
            fseek(f,sizeof(Estudiante)*-1,SEEK_CUR);

            // lo marcamos como dado de baja y lo grabamos
            e.bajaLogica = 1;
            fwrite(&e,sizeof(Estudiante),1,f);
        }

        // leo el siguiente
        fread(&e,sizeof(Estudiante),1,f);
    }

    fclose(f);

    return 0;
}

```

Ahora, veamos un programa que muestra todos los estudiantes del archivo, ignorando aquellos que fueron dados de baja.

```
int main()
{
    FILE* f = fopen("estudiantes.dat", "r+b");
    Estudiante e;

    // leo el primer estudiante
    fread(&e, sizeof(Estudiante), 1, f);
    while( !feof(f) )
    {
        if(e.bajaLogica==0 )
        {
            cout << estudianteToString(e) << endl;
        }

        // leo el siguiente
        fread(&e, sizeof(Estudiante), 1, f);
    }

    fclose(f);

    return 0;
}
```

En este ejemplo, el campo `bajaLogica` de la estructura `Estudiante` permite distinguir (marcar) aquellos registros que ya no forman parte del archivo porque fueron eliminados. Si la estructura no tuviera un campo especialmente preparado para este fin, podríamos establecer nuestro propio criterio de *marca de baja lógica*; por ejemplo, asignando un valor negativo en el campo `legajo` del estudiante.

3.1.2.12. Modificar registros

Para modificar cualquier registro de un archivo debemos posicionarnos en su byte inicial y luego reescribir el registro completo con `fwrite`, tal como lo hicimos en el apartado anterior, donde dimos de baja registros modificando su campo `bajaLogica`.

3.1.2.13. Longitud de un archivo

Para averiguar cuál es la longitud del archivo con el que estamos trabajando usamos una combinación entre las funciones `fseek` y `ftell`; esta última retorna el valor actual del indicador de posición del archivo.

La lógica será la siguiente:

1. Guardamos la posición actual del archivo, invocando a `ftell`.
2. Nos movemos al final del archivo, para lo cual invocamos a `fseek` pasándole el número de byte 0 (cero) y constante `SEEK_END`.
3. Guardamos la posición actual del archivo (es decir, la posición final).
4. Regresamos el indicador de posición al valor que guardamos en (1).

```
// :
int actual = ftell(f);
fseek(f,0,SEEK_END);      // me muevo al final
int ultimo = ftell(f);
fseek(f,actual,SEEK_SET); // regreso a donde estaba
int size = ultimo;
```

3.1.2.14. Cantidad de registros

La cantidad de registros que contiene un archivo de registros de longitud fija se puede calcular como el tamaño del archivo (expresado en bytes) dividido el tamaño del tipo de dato de sus registros.

3.1.2.15. Restricciones

Las operaciones de lectura y escritura sobre archivos se consideran costosas en términos de procesamiento y tiempo de ejecución, pues conllevan movimientos mecánicos que las tornan ineficientes en comparación con otro tipo de acciones, como ser el acceso a la memoria RAM.

De lo anterior se desprende la necesidad de reducir el uso de estas operaciones tanto como sea posible. Por tal razón, aceptaremos las siguientes restricciones relacionadas con la gestión de archivos:

1. Salvo en situaciones específicas y muy bien justificadas, de ningún modo nos permitiremos recorrer un archivo más de una vez.

2. Si el archivo es relativamente pequeño, podemos subir sus registros a memoria (por ejemplo, a una colección) y así disponer de ellos para consultarlos, ordenarlos, o realizar cualquier otro tipo de operación.
3. Los archivos no se ordenan, pues esta operación requeriría recorrerlos varias veces, lo que quedó restringido en (1). La única posibilidad viable sería la planteada en (2), o utilizar ciertas técnicas que estudiaremos más adelante.

3.1.3. Actividad práctica: API para el tratamiento de archivos de registros



Para simplificar el modo en que accedemos a los archivos, y proveer una visión de *archivo de registros* (no de bytes), desarrollaremos una API cuyas funciones estarán orientadas a la manipulación de registros de longitud fija.

El desarrollo de estas funciones queda a cargo del lector, pero una de ellas, a modo de ejemplo, la documentaremos y desarrollaremos aquí mismo.



API para el tratamiento de archivos de registros

La función `write`, cuyo prototipo es: `void write(FILE* f, T t)`, graba el valor `t` en el archivo `f`, siendo éste un archivo de registros tipo `T`, de longitud fija.

Un ejemplo de uso de esta función sería el siguiente:

```
FILE* f = fopen("demo.x", "w+b");
write<int>(f, 1);
write<int>(f, 2);
write<int>(f, 3);
fclose(f);
```

Para su desarrollo usaremos la función de biblioteca `fwrite`.

```
template<typename T>
void write(FILE* f, T t)
{
    fwrite(&t, sizeof(T), 1, f);
}
```

3.1.4. Ejemplos de uso de las funciones de la API

Usaremos los *templates* de la API que programamos durante el apartado anterior para desarrollar los mismos ejemplos que estudiamos a lo largo del capítulo.

3.1.4.1. Escribir y leer caracteres

```
// abro el archivo para
FILE* f=fopen("c.xx","w+b");

write<char>('A',f);
write<char>('B',f);
write<char>('C',f);
write<char>('D',f);

// cierro el archivo
fclose(f);
```

```
// abro para lectura
FILE* f=fopen("c.xx","r+b");

char c = read<char>(f);
while( !feof(f) )
{
    cout << c << endl;
    c = read<char>(f);
}

// cierro el archivo
fclose(f);
```

3.1.4.2. Escribir y grabar números enteros (*short*)

```
// abro el archivo para
FILE* f=fopen("s.xx","w+b");

write<short>(1234,f);
write<short>(4321,f);
write<short>(-9876,f);

// cierro el archivo
fclose(f);
```

```
// abro para lectura
FILE* f=fopen("s.xx","r+b");

short s = read<short>(f);
while( !feof(f) )
{
    cout << s << endl;
    s = read<short>(f);
}

// cierro el archivo
fclose(f);
```

3.1.4.3. Escribir y leer estructuras (*Persona*)

```
// abro el archivo para
FILE* f=fopen("p.xx","w+b");
```

```
// abro para lectura
FILE* f=fopen("s.xx","r+b");
```

```

Persona p = {10,"Pablo"};
write<Persona>(f,p);

p = {20,"Juan"};
write<Persona>(f,p);

p = {30,"Pedro"};
write<Persona>(f,p);

// cierro el archivo
fclose(f);

```

```

Persona p= read<Persona>(f);
while( !feof(f) )
{
    cout<<personaToString(p);
    p = read<Persona>(f);
}

// cierro el archivo
fclose(f);

```

3.1.4.4. Baja lógica

```

int main()
{
    FILE* f = fopen("estudiantes.dat","r+b");

    Estudiante e = read<Estudiante>(f);
    while( !feof(f) )
    {
        if( fechaAniosTranscurridos(e.fecUltExamen)>=2 )
        {
            // tenemos que eliminar a este estudiante
            int pos = filePos<Estudiante>(f)-1;
            seek<Estudiante>(f,pos);

            // lo marcamos como dado de baja y lo grabamos
            e.bajaLogica = 1;
            write<Estudiante>(f,e);
        }

        // leo el siguiente
        e = read<Estudiante>(f);
    }

    fclose(f);

    return 0;
}

```

3.1.5. Autoevaluación y ejercicios



Autoevaluación



Ejercicios

3.2. Lección 11

En esta lección veremos cómo realizar operaciones entre los bits que componen a los números enteros. Aunque este tema no está directamente relacionado con la gestión de archivos, la decisión de incluirlo en este capítulo obedece a que nos permitirá avanzar sobre algunos ejercicios que considero especialmente interesantes, los cuales sí vinculan ambos temas.

3.2.1. Operadores de bit

Los bits que componen a los bytes de las variables de cualquier tipo entero (`char`, `short`, `int` y `long`) pueden ser manipulados a través de un conjunto de operadores, que básicamente son de dos tipos: lógicos o de desplazamiento. Al aplicar cualquiera de estos operadores sobre un valor entero obtendremos otro valor del mismo tipo, que será el resultado de mover a izquierda o derecha los bits del valor original, o bien alterar sus valores mediante alguna operación lógica.



Lección 11

3.2.1.1. Operadores de desplazamiento

Los *operadores de desplazamiento* permiten mover hacia la izquierda o derecha los bits del valor de una variable numérica entera.

En el código que vemos más abajo, asignamos a `b` el valor que resulta al desplazar los bits de `a` una posición a la izquierda. Sería como si le “inyectásemos” al valor de `a` un bit 0 por la parte derecha. Esto desplazaría los otros bits a la izquierda, dejando fuera de la variable al bit más significativo.

```
unsigned short a = 3532; // a => |00001101|11001100|
unsigned short b = a<<1; // b => |00011011|10011000| => 7064
unsigned short c = b>>3; // c => |00000011|01110011| => 441
```

Analizando el ejemplo paso a paso, sería así:

Asignamos a a el valor 3532, que equivale al siguiente número binario.

a = |00001101|11001100|

Luego, le asignamos a b el resultado de: $a \ll 1$. Le “Inyectamos” un 0 por la derecha.

b = ⊕|00011011|10011000|0←

Finalmente, asignamos a c el valor resultante de desplazar los bits de b tres lugares a la derecha. Esto, visualmente sería como inyectar tres bits 0 por la parte izquierda, dejando fuera de la variable los 3 bits menos significativos.

c = →|00000011|01110011|000

3.2.1.2. Bases numéricas 8 (octal) y 16 (hexadecimal)

Podemos expresar cualquier valor numérico entero en base 8 o 16. Lo cual resulta muy conveniente cuando realizamos operaciones binarias, ya que estas bases numéricas ayudan a reducir considerablemente la cantidad de dígitos de un número binario; pues, representan con un dígito octal o hexadecimal combinaciones de 3 o 4 bits, respectivamente.

Por ejemplo: el número unsigned int a=755835234, que en binario es:

a = |00101101|00001101|00100001|01100010|

Podemos expresarlo en base 8 tomando grupos de 3 bits, de derecha a izquierda, y completarlo con ceros a la izquierda si fuera necesario.

```
101101000011010010000101100010
5  5  0  3  2  2  0  5  4  2
```

Este valor octal podemos asignarlo a una variable unsigned int anteponiéndole el prefijo 0 (cero), que indica que se trata de un valor octal.

```
unsigned int b = 05503220542;
```

El mismo análisis, pero tomando los bits en grupos de 4 nos dará una representación hexadecimal del número binario.

```
00101101000011010010000101100010
2   D   0   D   2   1   6   2
```

Para asignar este valor hexadecimal a una variable `unsigned int` debemos anteponer el prefijo `0x` (cero equis).

```
unsigned int b = 0x2D0D2162;
```

3.2.1.3. Operadores lógicos

Los operadores lógicos `&` (*and*) y `|` (*or*) permiten realizar operaciones lógicas entre los bits que componen a los números enteros, considerando que 0 (cero) equivale al valor lógico *false* y 1 (uno) al valor *true*.

La operación se realiza alineando en columnas los bits de los operandos y aplicando el operador lógico a cada columna, de derecha a izquierda.

Veamos un ejemplo del operador `&` (*and*).

```
unsigned short a = 3532;    // a => |00001101|11001100|
unsigned short b = 0x0F00;  // b => |00001111|00000000|
unsigned short c = a&b;     // c => |00001101|00000000| => 3328
```

En este ejemplo, le asignamos a `c` el valor que resulta de la operación `a&b`. Dicho valor se obtiene colocando 0 (cero) en aquellas columnas donde exista al menos un 0, y 1 (uno) donde ambos bits de la columna son 1.

```
a = 3532;    => |00001101|11001100|
b = 0x0F00;  => |00001111|00000000|
-----
c = a&b;     => |00001101|00000000|
```

Ahora el operador `|` (*or*)

```
unsigned short a = 3532;    // a => |00001101|11001100|
unsigned short b = 0x0F00;  // b => |00001111|00000000|
unsigned short c = a|b;     // c => |00001111|11001100| => 4044
```

Este operador asigna 1 (uno) en aquellas columnas donde al menos uno de los bits es 1, y 0 (cero) donde ambos valores de la columna también lo son.

```
a = 3532;    => | 00001101 | 11001100 |
b = 0x0F00; => | 00001111 | 00000000 |
-----
c = a&b;     => | 00001111 | 11001100 |
```

3.2.1.4. Máscara de bit

La máscara de bits es una técnica que permite ocultar parte de un número binario para exponer sólo aquellos bits que queremos ver.

Por ejemplo, sea la variable `unsigned int a = 755835234` que equivale al número binario: 00101101000011010010000101100010. ¿Qué valor decimal representan los bits de `a` pintados de color turquesa?

```
a    => | 00101101 | 00001101 | 00100001 | 01100010 |
```

Para averiguarlo debemos construir un número binario que tenga 1 (uno) en los lugares que queremos exponer (pintados de color turquesa) y 0 (cero) en los que queremos ocultar. A este número binario lo llamaremos *máscara de bit*.

```
mask => | 00000000 | 00011111 | 11110000 | 00000000 |
```

Esta máscara no es más que un valor numérico entero que construimos en función de los valores de sus bits.

Como C++ no permite representar números enteros en sistema binario, tendremos que convertirlo a otro sistema numérico que sí sea soportado por el lenguaje: decimal, octal o hexadecimal. Este último es el más sencillo.

```
mask => | 0  0 | 1  F | F  0 | 0  0 |
```

Ahora, declaramos la variable `mask` y le asignamos el número hexadecimal con la máscara de bits que diseñamos. Podemos omitir los ceros a la izquierda:

```
unsigned int mask = 0x1FF000;
```

El siguiente paso será realizar la operación lógica *and* entre `a` y `mask`.

```
unsigned int x = a&mask;
```

Analicemos bit por bit el resultado de esta operación:

```

a      =>|00101101|0001101|00100001|01100010|
mask  =>|00000000|00011111|11110000|00000000|
-----
x      =>|00000000|00001101|00100000|00000000|

```

La máscara anuló (dejó en cero) todos aquellos valores que no queríamos ver y conservó los que sí queríamos mantener (pintados de color turquesa).

Sólo resta mover los bits útiles (de color turquesa) hacia la derecha.

```
unsigned int resultado = x>>12;
```

La variable `resultado` internamente se verá del siguiente modo:

```
resultado =>|00000000|00000000|00000000|11010010|
```

Finalmente, los bits de `a` que pintamos de color turquesa son los mismos que aparecen en `resultado` alineados a la derecha. Todos los demás bits de `resultado` son 0 (cero). Por esta razón `resultado` coincide con el valor de los bits de `a` que queríamos averiguar.

```
cout << res << endl; // SALIDA: 210
```

3.2.2. Actividad práctica: TAD BitWriter y BitReader



Con los conocimientos que adquirimos durante esta lección, desarrollaremos dos TAD para escribir y leer *archivos de bits*. Sí, en este caso no pensaremos en archivos de registros ni de bytes. `BitWriter` y `BitReader` nos darán una visión binaria de los archivos que, si bien no es algo frecuente en el mundo de la programación, será de gran utilidad cuando, en el capítulo 6, desarrollemos la aplicación final.



*BitWriter y BitReader,
TAD para archivos de bit*

Si el lector no se siente en condiciones de abordar esta práctica, puede pasarla por alto, y retomarla antes de leer el mencionado capítulo.

Demás está decir que no es posible escribir *un bit* en un archivo, ni tampoco leerlo. Por esto, ambos TAD deberán implementar un mecanismo de *buffer*. Analizaremos brevemente la lógica de cada uno.

`BitWriter`, a través de su función `bitWriterWrite`, debe *recordar* cada bit que le mandamos a grabar. Así hasta juntar un conjunto de 8 bits, los cuales podrá grabar en el archivo, como si se tratase de un byte.

Análogamente, la función `bitReaderRead` del TAD `BitReader`, debe leer un byte desde el archivo, y *recordarlo* para ir entregando, *de a un bit*, cada vez que dicha función sea invocada. Así hasta entregar el último bit del byte leído. Luego, será el momento de leer el siguiente byte, y así sucesivamente.

A modo de ejemplo, analizaremos dos programas. El de la izquierda, escribe en un archivo la secuencia de bits: 01000001, que corresponde al valor numérico 65 (el carácter 'A'). El de la derecha, lee *bit a bit* dicho carácter.

```
FILE* f = fopen("x.x", "w+b");

BitWriter bw = bitWriter(f);
bitWriterWrite(bw, 0);
bitWriterWrite(bw, 1);
bitWriterWrite(bw, 0);
bitWriterWrite(bw, 0);
bitWriterWrite(bw, 0);
bitWriterWrite(bw, 0);
bitWriterWrite(bw, 0);
bitWriterWrite(bw, 1);

fclose(f); // 01000001
```

```
FILE* f = fopen("x.x", "r+b");

BitReader br = bitReader(f);
int bit = bitReaderRead(br);

while( !feof(f) )
{
    cout << bit; // 01000001
    bit = bitReaderRead(br);
}

fclose(f);
```

3.2.3. Autoevaluación y ejercicios



Autoevaluación



Ejercicios

3.3. ¿Qué sigue?

En el próximo capítulo nos dedicaremos a resolver problemas; pues ya disponemos de todas las herramientas necesarias: archivos, colecciones de objetos, y una sólida base de conocimientos sobre programación estructurada.